

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a vast, blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene and expansive feel.

DL for ENG

LECTURE 4.1

From Perceptron to Neural Network

Remus Teodorescu

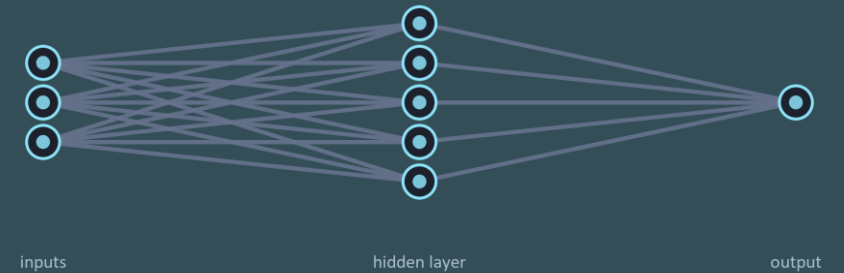
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

linear regression · XOR · ReLU · universal approximation

ONE OBJECT, BUILT IN FOUR STEPS



Everything in this course is this picture with different numbers in it. By the end of today you will know what every circle and every line means, and how many of each you need.

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **write down** — a linear model, a perceptron and a one-layer network, and say what separates them
- **explain** — why no setting of the weights solves XOR, and what a hidden layer changes
- **state** — what a ReLU network actually computes, and why that matters in Part 2
- **choose** — an activation for a given task, and defend the choice
- **size** — a network for an engineering problem, and justify starting small
- **recognise** — the three things this lecture deliberately left out

WHAT TODAY ASSUMES

from L3	the three ingredients
from L1–L2	Python, NumPy, a little PyTorch
not needed	any calculus beyond a derivative

ONE EQUATION CARRIES THE HOUR

A weighted sum, a bias, and a function applied to the result. Everything today is that, repeated.

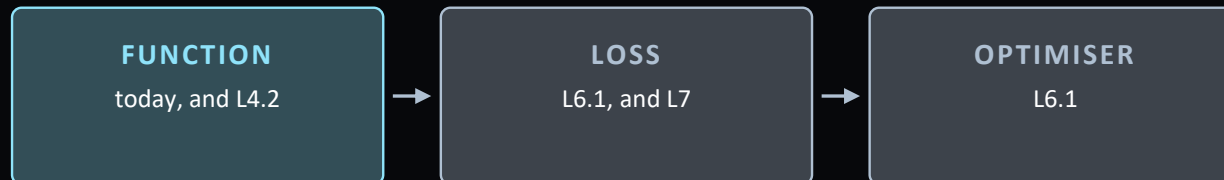
AND ONE THING IS MISSING

How the parameters are found. That is L6.1, and slide 23 lists what else we owe you.

The Three Ingredients, Made Concrete

TODAY THE FIRST ONE STOPS BEING ABSTRACT

- **three ingredients** — a function, a loss, an optimiser. L3.1 left them abstract.
- **today is the first one** — what is the function, and how much can it represent?
- **the loss stays simple** — sum of squared errors, as in your first year
- **the optimiser stays a black box** — until L6.1 opens it



Three ingredients. This lecture holds two of them fixed on purpose.

THE ROUTE THROUGH TODAY

- 1 a straight line, fitted
- 2 one neuron — and what breaks it
- 3 one hidden layer — the fix
- 4 what one layer can represent
- 5 and what the theorem does not promise



Note where today's model sits. We are at the data end of the spectrum all lecture, and that is the honest place to start.

Supervised Learning, Stated Properly

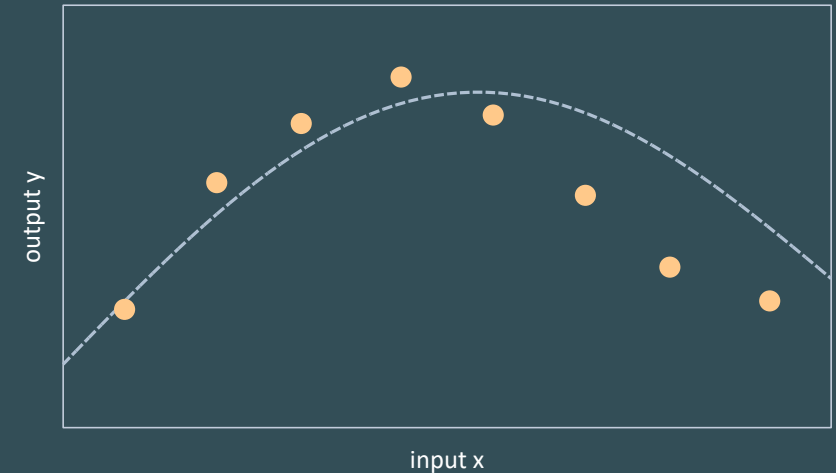
FOUR OBJECTS, AND NOTHING ELSE

- **N input-output pairs** — that is the whole setting
- **a model f** — takes an input and a parameter vector, returns a prediction
- **a loss** — scores how wrong the predictions are, in one number
- **every engineering choice** — is a choice of f , or of the loss. The rest is machinery.
- **training once, inference forever** — and that asymmetry is the entire commercial case for surrogate models

$$\hat{\theta} = \operatorname{argmin}_{\theta} \sum_{k=1}^N \ell(f(x_k, \theta), y_k)$$

$\hat{\theta}$ the fitted parameters, f the model, ℓ the loss, (x_k, y_k) the k th of N pairs

THE PICTURE



Amber dots are what you measured. The dashed line is the function you wish you had. Everything today is about how flexible f has to be to get close to it — and how flexible is too flexible.

A Straight Line, and Two Knobs

YOU HAVE DONE THIS SINCE YOUR FIRST YEAR

- **one-dimensional linear regression** — two parameters, intercept and slope
- **least squares** — minimise the summed squared vertical distances
- **that is the whole model** — and it is the baseline everything today is measured against

$$f(x, \theta) = \theta_0 + \theta_1 x$$

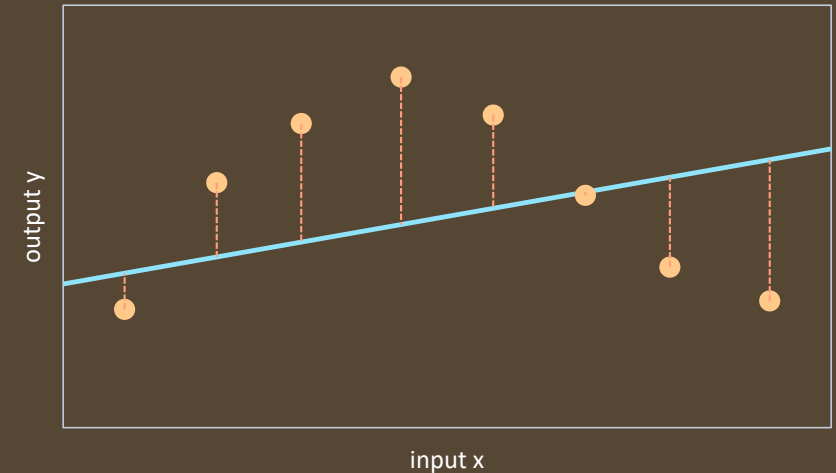
x the input, f the prediction, θ_0 the intercept, θ_1 the slope

$$\mathcal{L}(\theta) = \sum_{k=1}^N (\theta_0 + \theta_1 x_k - y_k)^2$$

$\mathcal{L}$ the loss, $\theta_0 + \theta_1 x_k$ from the model, y_k from the label, N data points

- **a complete instance of all three ingredients** — and one you already trust

THE FIT, AND ITS RESIDUALS



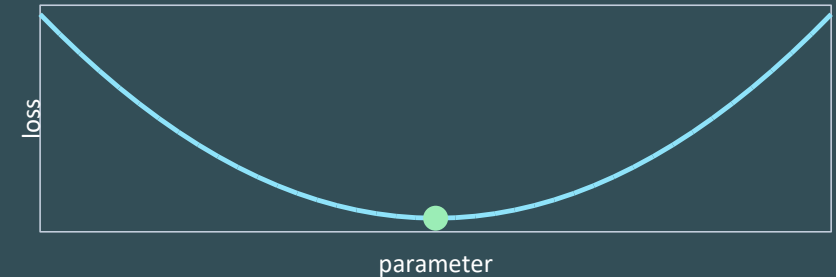
Orange dashes are the residuals. Least squares makes the sum of their squares as small as two knobs allow — which here is not very small at all.

The Loss Is a Landscape

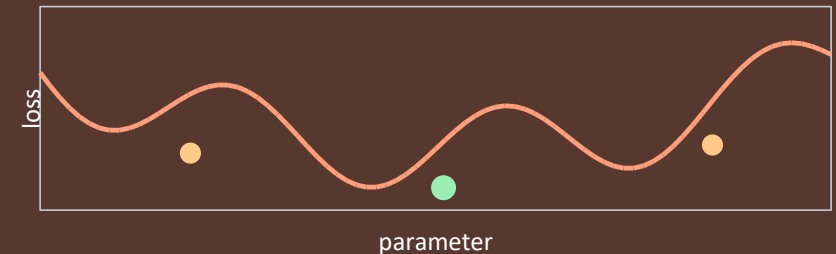
A PICTURE WORTH CARRYING TO LECTURE SIX

- **sweep the parameters** — and the loss traces a surface over them
- **for least squares** — that surface is a perfect bowl. One minimum, closed form.
- **no search needed** — the minimum has a closed form you can write down
- **this stops being true** — the moment the model is a network
- **which is why L6 exists** — and it is the key structural difference

LINEAR MODEL — ONE MINIMUM



A NEURAL NETWORK — MANY MINIMA



The Perceptron: One Neuron, Entire

THE MACHINE FROM LECTURE THREE, AS MATHEMATICS

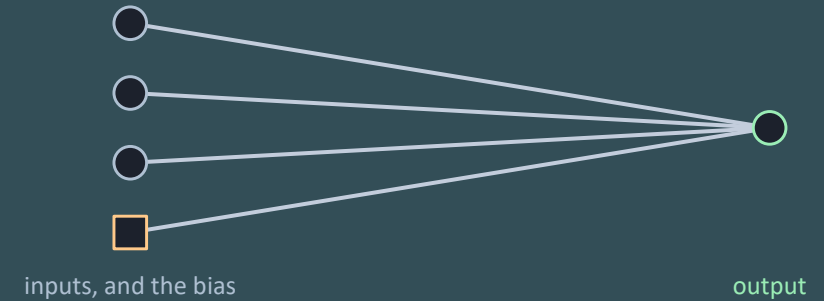
- weighted sum, plus a bias, through a step function
- output 1 if the sum is positive, 0 otherwise
- **that is Rosenblatt's machine** — and it is one line of algebra

$$y = H(\mathbf{w} \cdot \mathbf{x} + b)$$

$\mathbf{x}$ the inputs, $\mathbf{w}$ the weights, b the bias, H the step function, $y \in \{0, 1\}$

- **it also learns** — show it an example: if right change nothing, if wrong nudge the weights
- **Rosenblatt proved it converges** — in finite steps if a line exists, and that is why it caused such excitement

ONE NEURON



WHAT THE THEOREM SAYS

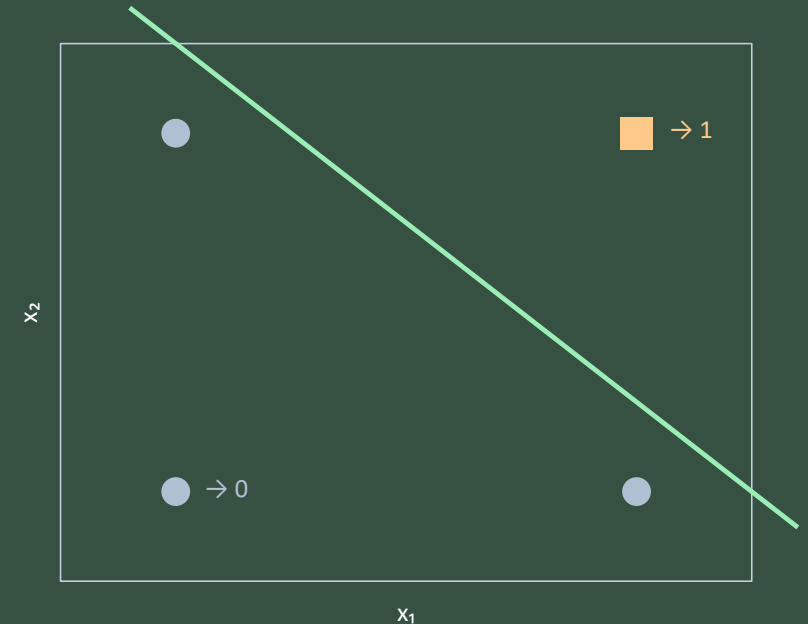
if	the classes are linearly separable
then	the rule converges, in finite steps
if not	it never converges — it cycles
nobody	checked the second line, in 1958

What One Neuron Can Do

A LINE THROUGH THE PLANE

- the weighted sum is zero along a line — positive one side, negative the other
- so a perceptron draws a straight boundary — nothing more
- **AND** — three zeros together, one apart. A ruler separates them.
- **OR too** — and anything whose classes fall either side of some line

AND — SEPARABLE



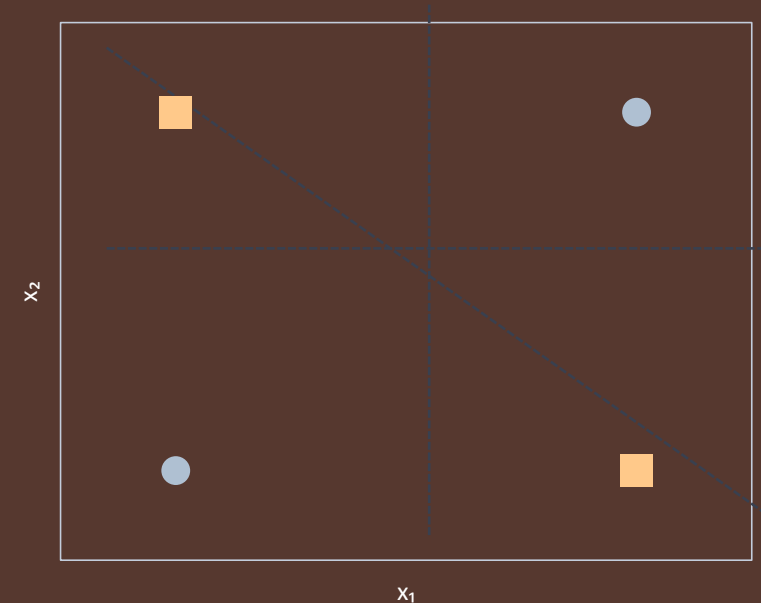
Amber square is class 1, grey circles are class 0. Many lines separate them — one is all you need.

Now Try Exclusive Or

FOUR POINTS. TWO CLASSES. TRY IT.

- **exclusive-or** — true when exactly one input is true
- **take thirty seconds** — look at the picture and try to draw the line
- **you cannot** — the two classes sit on opposite diagonals
- **not a flaw in the learning rule** — no weights represent this function at all

XOR — NOT SEPARABLE



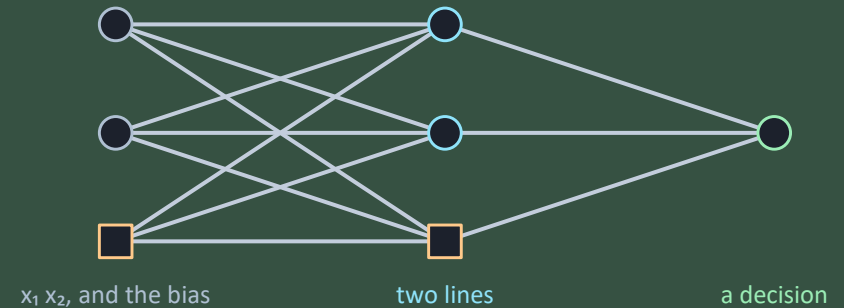
Three candidate lines, all wrong. Every straight line misclassifies at least one point.

The Fix: One More Layer

TWO LINES, THEN A DECISION ABOUT THEM

- **add a hidden layer** — two neurons, each drawing its own line
- **nothing new invented** — the same operation twice, with a nonlinearity in between
- **the output neuron** — draws a line in the space of the two hidden decisions
- **and there the problem is separable** — the hidden layer bent the plane
- **Minsky and Papert knew this in 1969** — what they could not say was how to train it

TWO INPUTS, TWO HIDDEN, ONE OUT



THE WHOLE OF DEEP LEARNING, ALMOST

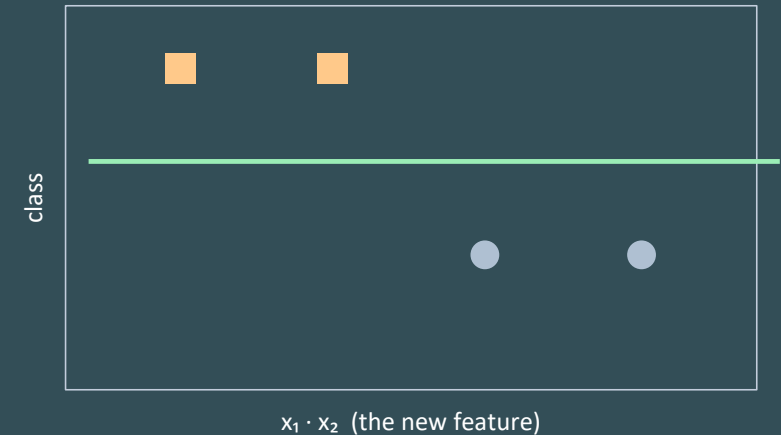
A layer transforms the space; the next layer solves an easier problem in that new space. Stack enough of them and a problem that needed a wildly complicated boundary in the original coordinates needs a simple one at the end.

Linear Separability, and Why It Matters to You

THE GENERAL STATEMENT

- **a single layer represents** — exactly what a hyperplane can separate
- **XOR is the smallest failure** — which is why it became the example
- **but separability depends on representation** — feed it $x_1 \cdot x_2$ and XOR is trivial
- **that was feature engineering** — the whole discipline until about 2012
- **the hidden layer learns the features** — which is why deep learning mattered

THE SAME PROBLEM, ONE EXTRA FEATURE



Add the product $x_1 x_2$ as a third input and XOR becomes separable by a single line. The function did not change. The representation did — and that is the whole of feature engineering in one picture.

Why There Must Be a Nonlinearity

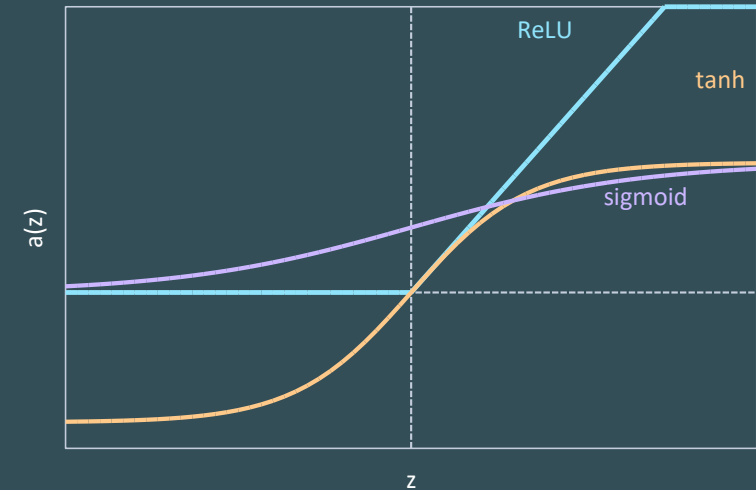
THE ONE-LINE ARGUMENT

- **without an activation** — a linear function of a linear function is linear
- **stack a hundred layers** — and you still have a straight line
- **so it is not a detail** — it is the only reason depth buys anything
- **the modern default is ReLU** — pass positives, zero the negatives
- **almost embarrassingly simple** — and it works

$$a(z) = \text{ReLU}(z) = \max(0, z)$$

z the weighted sum reaching the neuron, $a(z)$ what the neuron passes on

THREE ACTIVATIONS



Vertical axis runs from -1 to 2 . ReLU is the default everywhere in L4 and L5. Note that it is not differentiable at zero — that is the reason for the next slide.

Why Every Network in L7–L12 Uses tanh

A CHOICE THAT LOOKS LIKE A DETAIL AND IS NOT

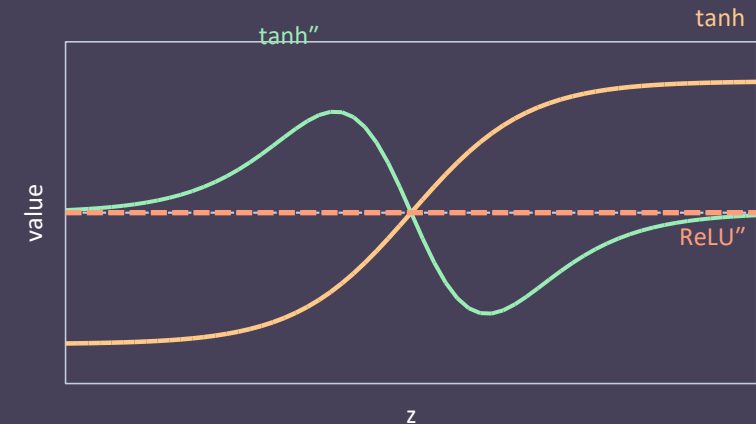
ReLU is the right default for everything in Part 1. It is the wrong default for everything in Part 2, and the reason is one line of calculus.

THE ARGUMENT

- **Part 2 needs second derivatives** — the PDE residual is built from them
- **ReLU is piecewise linear** — second derivative zero, so the physics term is meaningless
- **tanh is smooth** — derivatives of every order exist and are non-trivial
- **that is why L7 onward uses tanh** — not habit, a requirement

$$\frac{d^2}{dz^2} \text{ReLU}(z) = 0 \quad \frac{d^2}{dz^2} \tanh(z) \neq 0$$

CURVATURE, OR THE LACK OF IT



The orange line is the second derivative of ReLU. It is zero.
That is the slide.

A Shallow Network, Written Out

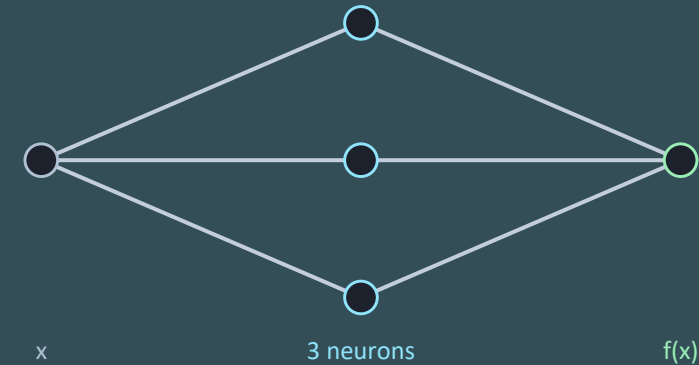
ONE HIDDEN LAYER, D NEURONS, IN ONE LINE

- **each hidden neuron** — scales the input, shifts it, applies the activation
- **the output** — a weighted sum of those D responses
- **two parameters per neuron** — plus one weight and a bias at the output
- **ten neurons is thirty-one parameters** — small by any standard

$$f(x, \theta) = \varphi_0 + \sum_{d=1}^D \varphi_d a(\theta_{d0} + \theta_{d1}x)$$

D hidden neurons, θ_{d0}, θ_{d1} the bias and weight of neuron d, φ_0, φ_d the output bias and weights

ONE INPUT, THREE HIDDEN, ONE OUTPUT



PARAMETER COUNT

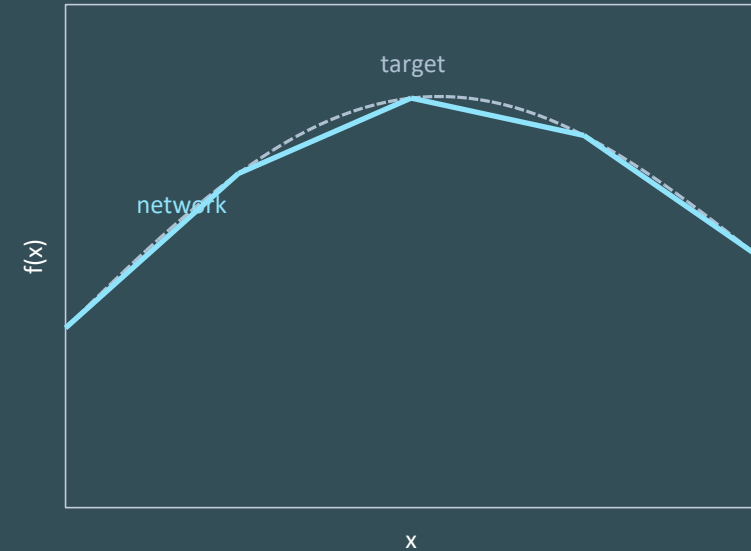
D = 3	10 parameters
D = 10	31 parameters
D = 100	301 parameters

Three Neurons, Four Linear Regions

THE MOST USEFUL PICTURE IN THE WHOLE BOOK

- **each ReLU neuron** — flat, then rising. One breakpoint each.
- **three neurons, three breakpoints** — four linear regions
- **not an approximation of the shape** — it is exactly what the network computes
- **training moves the breakpoints** — that is what the parameters do

$D = 3$



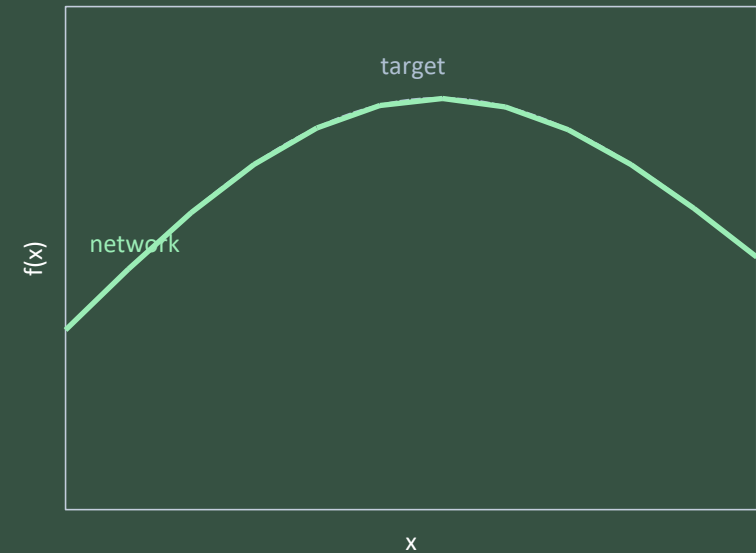
*Four linear regions, three breakpoints, ten parameters.
Recognisably the right shape and obviously not the right
function.*

Ten Neurons, and the Curve Appears

STILL PIECEWISE LINEAR — JUST WITH MORE PIECES

- ten neurons, eleven linear regions, thirty-one parameters
- **from a distance, a smooth curve** — up close, still straight lines
- **that is the honest description** — piecewise linear, pieces placed by training
- **and why slide 12 is not a technicality** — there is genuinely no curvature
- **so what does a hundred neurons get?** — and is there a limit?

D = 10



Thirty-one parameters. You will reproduce this exact experiment in Ex_04 notebook 03, sweeping D and watching the error fall.

Universal Approximation

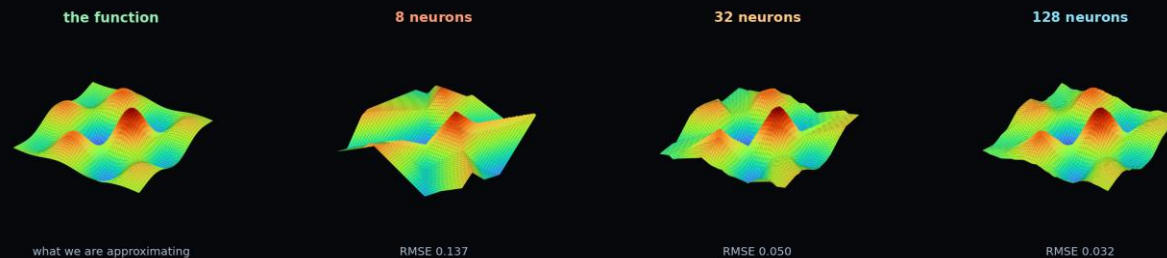
THE STATEMENT, PRECISELY

For any continuous function on a closed bounded region, and any tolerance you name, there exists a network with one hidden layer and enough neurons whose output is within that tolerance everywhere on the region. Cybenko 1989, Hornik 1991.

WHY IT IS TRUE, INFORMALLY

- **you have already seen the argument** — enough linear regions approximate any curve
- **the theorem says** — one hidden layer suffices given enough neurons, and says nothing about how many

Two inputs here, not one: each neuron folds the plane instead of bending a curve.



AND WHAT IT DOES NOT SAY

- not how many** more neurons than you could train
- not findable** existence, not a way to find it
- not from data** it assumes you know the function
- not efficient** one layer may need exponentially more
- not stable** nothing about behaviour off the region

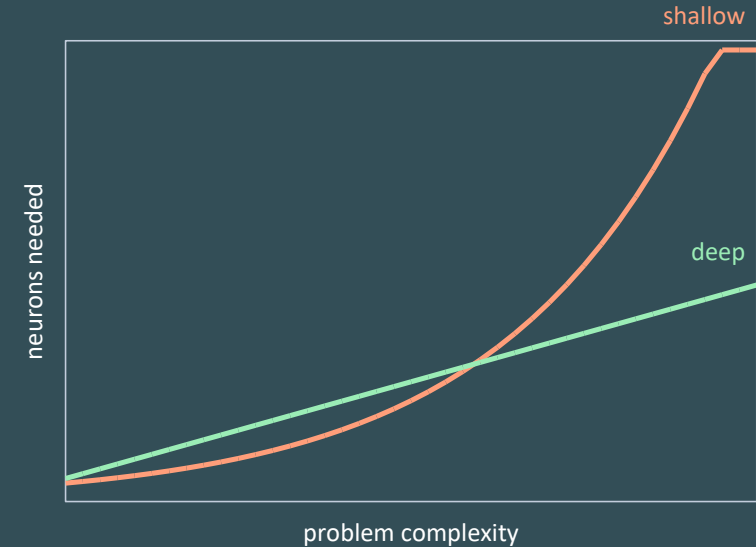
Every one of those five is the subject of a later lecture. The theorem is the beginning of the question, not the end of it.

If One Layer Is Enough, Why Go Deep?

THE ANSWER IS ABOUT COST, NOT CAPABILITY

- **universal approximation is silent on cost** — and the cost can be severe
- **some functions** — deep needs neurons growing linearly, shallow exponentially
- **the intuition is slide 9** — a deep network reuses what earlier layers computed
- a shallow one rebuilds everything from the input
- **that is all of L4.2** — and why depth is the default, even when the problem is small

NEURONS NEEDED, SCHEMATIC



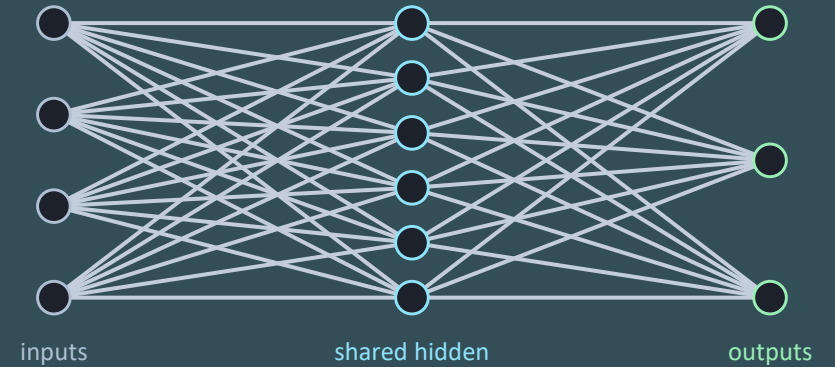
Schematic, not measured — the axes have no units on purpose. The claim is about the shape of the two curves and nothing more.

More Inputs, More Outputs

NOTHING CONCEPTUAL CHANGES

- **several inputs** — the breakpoint becomes a hyperplane, not a point
- **several outputs** — several neurons reading one shared hidden layer
- **that sharing matters** — in L9 one body outputs velocity and pressure together
- **and the count grows** — $D(D_i+1) + D_o(D+1)$, quickly

FOUR IN, THREE OUT



WHERE YOU WILL SEE THIS

L9: one network outputs the two velocity components u and v and the pressure p from a shared hidden layer, and the coupling between them is not incidental — it is why the physics residual can be written at all.

The Words, Settled Once

SO THAT NOTHING LATER IS AMBIGUOUS

- **the vocabulary is inconsistent** — and occasionally misleading; this course uses the words below, consistently
- **the neurons have continuous activations** — not step functions

neuron / unit	one weighted sum plus an activation
layer	a set of neurons reading the same inputs
hidden layer	any layer that is neither the input nor the output
width	neurons in a layer
depth	number of hidden layers
weights and biases	the parameters — what training changes
activation	the nonlinearity inside a neuron

NAMES THAT MISLEAD

multi-layer perceptron	contains no perceptrons
neural network	no brain involved, since 1960
deep	many layers, nothing more
learning	fitting parameters
hidden	not observed, not magic
neuron	a borrowed word, not a cell

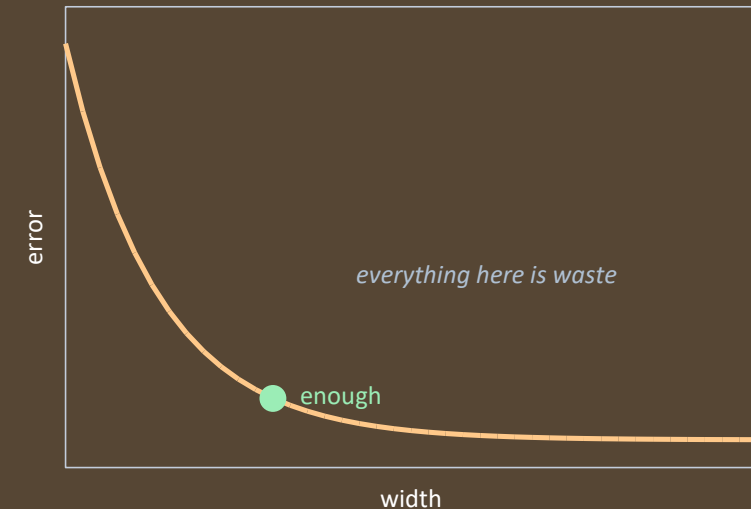
Every one of these has caused a confused question in a previous year, so none of them is on this slide by accident.

How Big a Network?

SMALLER THAN YOU THINK

- **every PINN in L7–L12** — four to eight layers, twenty to fifty neurons
- **not modesty** — the network represents one smooth field on one domain
- start small, grow when it demonstrably cannot fit
- **underfitting announces itself** — overfitting does not
- **bigger is not better here** — whatever the headlines say

ERROR AGAINST WIDTH, SCHEMATIC



The curve flattens. Past the elbow you are paying for parameters that buy you nothing and cost you training time and generalisation.

Failure Modes of a First Network

IT PREDICTS A CONSTANT

The most common first result. Every output is the mean of the training targets. Usually the learning rate, sometimes unscaled inputs, occasionally a dead activation — and it looks like the model 'did not learn', which is not a

THE LOSS IS NaN

Almost always scaling. Inputs in the thousands and targets in the millionths, multiplied together through several layers, leave the range of a float. Normalise both, and the problem disappears.

IT FITS PERFECTLY AND IS USELESS

Training error near zero, and nonsense on anything new. This is overfitting, and it is the one that survives to a report because nothing in the training output warns you.

WHAT TO CHECK, IN ORDER OF COST

free	print the input and target ranges
free	check the loss actually decreases
cheap	overfit five samples on purpose
cheap	drop the learning rate tenfold
real work	hold out data and look at it

The third one is the best debugging trick in this course. A network that cannot memorise five points has a bug; a network that can, but will not fit the full set, has a training problem. Those need completely different fixes.

Ex_04 — Build It Yourself

FROM NUMPY TO PYTORCH, IN ONE SITTING

- **notebook 01 is the one that matters** — a perceptron in plain NumPy
- **watch it solve AND** — then watch it fail on XOR
- **add a hidden layer** — two neurons, nine parameters, and it works
- **notebook 02** — the same thing in PyTorch, to see how much was bookkeeping
- **notebook 03** — reproduce the linear-region experiment yourself



THE NOTEBOOKS

00	environment check — read only
01	perceptron and XOR, in NumPy
02	the same network in PyTorch
03	counting regions — sweep D
04	depth against width
05	the report

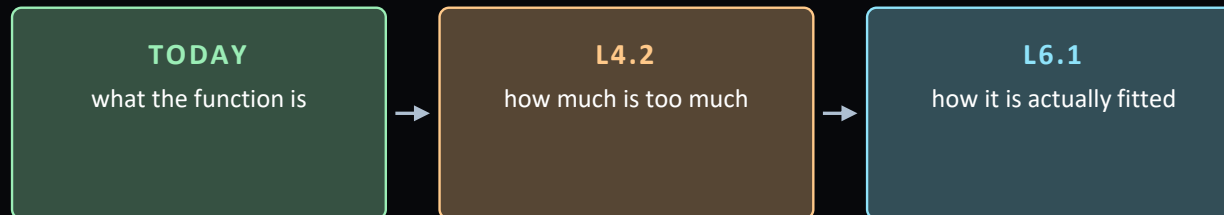
THE QUESTION THAT CARRIES THE MARKS

At equal parameter count, which did better — deeper or wider? And why do you think so?

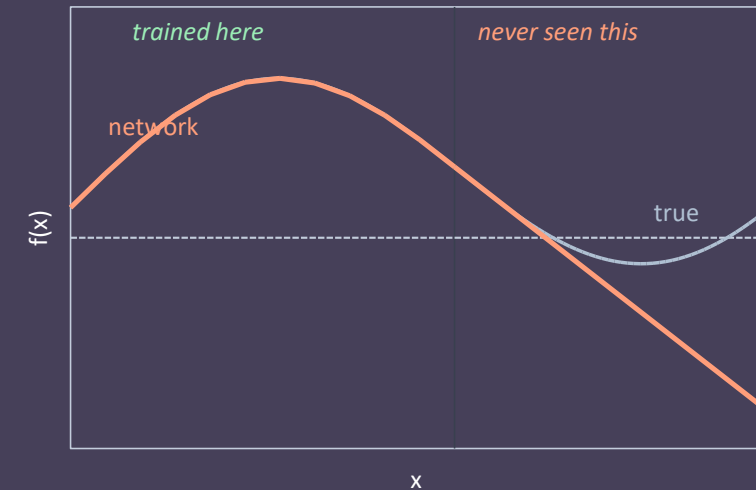
What This Lecture Quietly Assumed

THREE THINGS I HAVE NOT JUSTIFIED

- **I fitted every curve today** — without saying how the parameters were found
- **that is L6.1** — gradient descent and backpropagation
- **I used squared error as if obviously right** — it is right under one assumption
- **and said nothing about unseen data** — which is all of L4.2



YOUR EX_03 PLOT, EXPLAINED



A piecewise-linear function outside its training range continues along whichever straight segment it ended on. That is not a defect — it is exactly what slide fifteen said it was.

Key Takeaways

- 1 A neuron is a weighted sum**
Plus a bias and a nonlinearity. Nothing about it is biological, and nothing about it is complicated.
- 2 XOR is why layers exist**
One neuron draws one line. A hidden layer bends the space so that a line is enough.
- 3 ReLU networks are piecewise linear**
Each neuron adds one breakpoint. Training moves them. That is the whole picture.
- 4 Universal approximation promises little**
Existence, not findability, not efficiency, not behaviour outside the region you trained on.
- 5 tanh, not ReLU, from L7 onward**
Because ReLU's second derivative is zero, and a PDE residual needs curvature.

Next — L4.2: what happens when you stack them, and how much capacity is too much.

Questions

TEN TO TEST WHETHER TODAY LANDED

- **why is the least-squares loss surface a bowl** — and what changes when the model is a network?
- a perceptron draws one line. What exactly does the hidden layer add?
- **XOR** — is the problem the learning rule, or the model?
- **give a representation** — in which XOR becomes linearly separable
- **what does a ReLU network compute** — stated exactly, not approximately?
- **why does removing the activation** — make depth worthless?

AND FOUR MORE

- seven** why tanh and not ReLU, from L7 on?
- eight** one layer suffices — so why go deep?
- nine** how would you size a network not yet built?
- ten** what did this lecture leave out?

THE ONE WITH NO SETTLED ANSWER

How would you know your network is too small, before you train it?

BEFORE L4.2

Run notebook 01. Bring the XOR failure with you.

References and Further Reading

THE TWO BOOKS

Prince, Understanding Deep Learning — chapter 2 for supervised learning and linear regression, chapter 3 for shallow networks. Chapter 3's figures are the source of the linear-region picture and are worth an hour on their own.

Goodfellow, Bengio & Courville — chapter 1.2 for the perceptron history and the two winters, chapter 6.1 for the XOR example worked in full algebra, which is a good check on your own Ex_04 solution.

PRIMARY, IF YOU WANT THEM

Rosenblatt (1958), The perceptron: a probabilistic model, Psychological Review 65 — readable, and startlingly ambitious.

Minsky & Papert (1969), Perceptrons — the XOR argument. Short, and much more careful than its reputation.

Cybenko (1989) and Hornik (1991) — the universal approximation results. Cite them accurately: they are existence theorems.

READ IN THIS ORDER

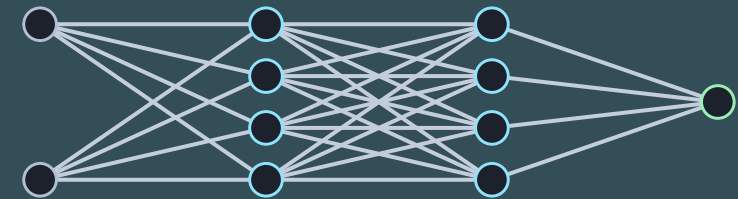
before Ex_04 UDL ch. 3.1 — the region figures

if XOR is unclear GBC ch. 6.1, worked in full

before L4.2 UDL ch. 4

for the exercise the run guide in Ex_04

only if curious Rosenblatt (1958), for the tone



L4.2 adds the middle column.

NEXT

L4.2 — Deep Neural Networks

BEFORE THE NEXT SESSION

read	UDL ch. 4 — about forty minutes
run	Ex_04 notebooks 00 to 02
bring	the epoch count where XOR failed
think about	why nine parameters were enough

Bring the number. L4.2 opens with a show of hands on how long people let the single-layer version run before believing it would never converge, and the range is always instructive.